Self-Paced Module: Complete GUI Application

Java Programming · Grade 12 TVL · Cielito Zamora Senior High School
Week 21 · December 7–11, 2026 · Continues Week 20
Name: _____ Section: _____ Date: _____

Day / time	Focus	Work / strip
Mon Dec 7 / 2 h	Limited design	Page 1 Activity 1; page 7 bottom.
Tue Dec 8 / 2 h	Integrate workflow	Page 2 Activity 2; page 7 middle.
Wed Dec 9 / 2 h	Working menus	Pages 3–4 Step 1 / Activity 3; page 7 top.
Thu Dec 10 / 3 h	Complete and repair	Pages 5–6 Activity 4; page 8 bottom.
Fri Dec 11 / 3 h	Verify and present	Page 6 Activity 5 / presentation; page 8 top.

How to use: Work offline in order with the printed module, JDK and existing sources. Keep each public class in its named .java file, all in one folder. Compile **javac *.java**; Vehicle runs with **java Main**, school-materials practice with **java RequestDemo**. Use your own launcher for assessed work. LIPLET means quiz strip. Error guide and school checks: page 5.
Starting files: Use the complete Week 20 starting files directly: they already include all four Week 19 steps. Do not reapply old steps. Vehicle: java Main, with the Week 18 launcher/readiness group and Week 19 JComboBox TerminalPanel. School-materials practice: java RequestDemo, with Preview, Info, Rename and confirmed Clear. Keep LayoutDemo, ThemeLayoutCheck, all other optional demonstrations, every .txt backup, the Vehicle model and exceptions, FareService/FareCheck, ReadyTask and DispatchThread, BoardingTask, ReadyReport, ConvoyLeg and LoadingBay. InvalidDistanceException extends RuntimeException directly; keep its separate catch.

MONDAY | Plan One Useful Application

Goals: Link a limited set of requirements and a user flow to the Week 20 own-theme proposal. **Value:** Make realistic promises and distinguish existing, planned, completed and unverified work. **Produce:** Draw an interface plan and write needs-linked requirements and checks for one useful task.

Definition | Complete classroom application. A GUI (graphical user interface) uses windows and controls to work with a program. A complete classroom application finishes one useful user task, checks input and explains its limits. A requirement is a behavior you can check; a user flow is the path from starting an action to a useful result [YS].

ANALOGY A school desk needs a clear path from choosing materials to a readable draft. More counters do not help if nobody can finish the request.

Continue your proposal. Name your own theme, users and problem. Reuse Week 20 findings and label assumptions. Existing means already in the program; planned means proposed; completed means built with stated checks; unverified means not yet checked. A completed feature may still have unverified usability.

Worked example and justified revision. Keep the school-materials draft. Week 20 planned local save/load, but did not build it. Defer that feature to protect time for safe integration and menus; retain the current one-session preview task. The hypothetical scenario, costs, benefits and estimates were not field research. No new survey is required.

Need	Limited requirement / check
Clear draft	R1: choose subject, at least one material and collection place; Preview gives a summary or a correctable error.
Recover safely	R2: Rename accepts 1–30 trimmed characters; Cancel keeps the name. Clear changes choices only after explicit Yes.
Find useful actions	R3: Request has Preview/Rename/Clear; View has Info. Each menu uses the same behavior as its button.

Worked flow: open → select subject/materials/place → Preview → read summary. On an error, correct the choice and Preview again. Rename is optional. Clear offers Yes/No/Cancel; closing its dialog also keeps choices. Preview is a snapshot: after editing choices, Preview again.

Activity 1 | Own-theme design. In your notebook, name your existing capability, user/task/problem and proposal link. Write 3–5 limited requirements with a need and a concrete check each. Draw nested panels and a menu plan; mark inputs, action, model method and result. Write the normal flow, correction path and cancellation path. Justify a reduced scope using Week 20 feasibility.

Boundary: the example does not submit, reserve stock, save/load or serve simultaneous users. Closing the window loses its draft. Use invented request labels, not personal data. Your own task must remain useful within its stated limits.

Own-theme planning space	Your notes
Theme / user / useful task	_____
Requirement → need → check	_____ _____
Proposal revision / reason	_____

Course history: Week 11 classes/objects; 12 four OOP pillars; 13 exceptions; 14 collections; 15 file handling; 16 threads; 17 Swing basics; 18 layouts/panels; 19 advanced components/user interaction; 20 market analysis/system proposal.
References: [1 YD], [2 YS], [3 YT], [4 MIT], [5 PA]. Links/excerpts: page 5. **Key validated facts:** Choose a user task and realistic boundaries [YD] [YS]; connect controls to application rules [MIT].
LIPLET | MONDAY: After Activity 1 on page 1, answer the MONDAY strip (bottom of page 7). Detach only today's strip.

TUESDAY | Combine a Complete User Workflow

Goals: Explain how nested panels, layouts, actions and existing model rules complete one workflow. **Value:** Respect users by preserving useful checks and making correction clear. **Produce:** Combine existing controls and model calls in the own-theme program and record valid and invalid trials.

Definition | Integration. Connecting controls, layouts, existing model rules and feedback so one user can finish a task. Nested panels are panels inside other panels; a layout arranges their controls. A model holds application data or rules; a listener is code called when an action occurs [MIT] [N1].

ANALOGY A materials desk connects its form, checking rule and readable result. A separate attractive form with no working route to the rule leaves the task unfinished.

Trace the retained RequestDemo. It already combines the controls and rules below. Today reuse that complete workflow and explain its structure; no replacement of the worked example is needed. Build or repair the equivalent connection in your own theme program.

Group / layout	Existing connection
RequestDemo / BorderLayout	choices NORTH; result CENTER; actions SOUTH.
choices / BorderLayout	subject label NORTH; scrollable JList CENTER; options SOUTH.
options / GridLayout(2,1)	materials row and collection-place row, each using FlowLayout.
Actions and rules	Preview/Info call summary(); it checks required choices. Rename uses acceptName(); Clear uses clearIfConfirmed().

Follow an action: button → actionPerformed() → summary() → useful text or IllegalArgumentException caught as feedback. This retained teaching example keeps request rules in methods of the same class. In your own theme, call the existing model method; do not duplicate its calculation in a listener. Vehicle already routes TerminalPanel through FareService to Vehicle.quoteFare(). Do not merge the two programs.

Read existing code, not a replacement: `result.setText(text);` displays the current summary; `summary()` throws if a required choice is missing. Keep those checks. A disabled or empty choice is not a reason to remove validation.

Actual direct-check results: with no subject: IllegalArgumentException: Choose a subject. With Java but no material: IllegalArgumentException: Choose materials. Java + Guide + Desk returns: Java: Guide=true; Sheet=false; Place=Desk. These came from method checks on the event dispatch thread, not live clicks. Headless means using no display: that launcher run showed no window.

Safe interface work. The event dispatch thread (EDT) handles Swing events and interface updates [KS]. Keep short checks there. Slow work runs outside listeners; return updates through SwingUtilities.invokeLater(). Retain Main's ReadyTask and Week 16 helpers. This draft preview has no slow operation, so it needs no new worker or storage.

Activity 2 | Own-theme workflow. Combine existing components and useful nested panels. Trace one action to its model method and visible result. Compile locally with javac *.java; run your own launcher. Record normal input, missing/invalid input, correction and cancellation. State expected and actual results separately. Back up each working file before changing it.

Check yourself: can a classmate complete the task from labels alone? Does invalid input preserve editable choices? At minimum and larger window sizes, can all controls still be reached? Keep the interface's result and your description consistent.

Activity 2 | Record your own trials

Input or action	Expected	Actual / status
Normal task	_____	_____
Invalid → correction	_____	_____
Cancellation	_____	_____

Connection record: control ____ → listener ____ → model method ____ → result ____.
Nested groups: parent/layout ____ contains child/layout ____.
Repair or remaining check: _____

Draw and check the own-theme workflow

Sketch labelled input, nested panels, action, model method and feedback. Add the correction and cancellation routes. Keep this with your test record.

References: [1 N1], [2 MIT], [3 KS], [4 PA], [5 YS]. Links/excerpts: page 5. **Key validated facts:** Group related controls and reuse application rules [N1] [MIT]; keep handlers short [KS].
LIPLET | TUESDAY: After Activity 2 on page 2, answer the TUESDAY strip (middle of page 7). Detach only today's strip.

WEDNESDAY | Add Purposeful Menus Safely

Goals: Explain JMenuBar, JMenu and JMenuItem and trace a menu action to existing behavior. **Value:** Protect user choices when an action is cancelled or declined. **Produce:** Build purposeful menus with keyboard access and test validation, correction and safe confirmation.

Definition | Menu hierarchy. JMenuBar is the menu strip, JMenu groups related choices, and JMenuItem is an action inside a menu. Its listener works like a button listener [N2].

ANALOGY Request and View are like labelled desk folders. Inside each folder, each instruction performs a real task rather than merely showing a decorative label.

Step 1 | Add menus to the worked example. First copy the previous working RequestDemo.java to RequestDemo_week20.txt. Keep every earlier backup. Then replace RequestDemo.java with the complete listing across pages 3–4: read left column, then right column on each page. Do not run half a file. No other existing file changes. Compile javac *.java; run java RequestDemo.

What changes: createMenuBar() adds Request → Preview/Rename/Clear and View → Info, using the same listener as the retained buttons. The frame installs that bar. Subject/material/place keyboard mnemonics and a scrollable result improve access. Existing validation, dialogs and safe confirmation remain. A mnemonic is a letter used to navigate controls; an accelerator runs an action directly: Ctrl+P previews [N2].

Before typing | trace the plan	Your prediction / later check
Request → Preview → listener	Which existing method checks the choices? _____
Clear No / Cancel / close	Which choices and name must remain? _____
Keyboard route	Request/View letters ____; Preview accelerator ____
Changed choices after preview	What must the user repeat for a current result? _____

Model the menu tree: Request → Preview, Rename, Clear; View → Info. A menu groups actions; the existing listener decides which rule to run. Do not add empty menus or rename an action without updating its command check.

RequestDemo.java · lines 1–46

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
public class RequestDemo extends JPanel
    implements ActionListener {
  private JCheckBox guide = new JCheckBox(
    "Guide");
  private JCheckBox sheet = new JCheckBox(
    "Practice sheet");
  private JRadioButton desk = new JRadioButton(
    "Desk", true);
  private JRadioButton room = new JRadioButton(
    "Classroom");
  private JList<String> subject = new JList<>(
    new String[]{"Java", "English", "Math"});
  private String name = "My request";
  private JTextArea result = new JTextArea(4, 24);
  public RequestDemo() {
    setLayout(new BorderLayout(8, 8));
    setBorder(BorderFactory.createEmptyBorder(8, 8,
      8, 8));
    guide.setMnemonic('G'); sheet.setMnemonic('S');
    desk.setMnemonic('D'); room.setMnemonic('L');
    ButtonGroup collect = new ButtonGroup();
    collect.add(desk); collect.add(room);
    JPanel materials = new JPanel();
    materials.add(guide); materials.add(sheet);
    JPanel place = new JPanel();
    place.add(new JLabel("Collect at:"));
    place.add(desk); place.add(room);
    JPanel options = new JPanel(new GridLayout(2,
      1));
    options.add(materials); options.add(place);
    subject.setSelectionMode(
      ListSelectionModel.SINGLE_SELECTION);
    subject.setVisibleRowCount(3);
    JLabel label = new JLabel("Subject:");
    label.setLabelFor(subject);
    label.setDisplayedMnemonic('U');
    JPanel choices = new JPanel(new BorderLayout(4,
      4));
    choices.add(label, BorderLayout.NORTH);
    choices.add(new JScrollPane(subject),
      BorderLayout.CENTER);
    choices.add(options, BorderLayout.SOUTH);
    result.setEditable(false);
```

RequestDemo.java · lines 47–92

```
    result.setLineWrap(true);
    result.setWrapStyleWord(true);
    result.setText(
      "Choose materials, then Preview.");
    JPanel actions = new JPanel();
    for (String text : new String[]{"Preview",
      "Clear", "Info", "Rename"}) {
      JButton button = new JButton(text);
      button.setMnemonic(text.charAt(0));
      button.addActionListener(this);
      actions.add(button);
    }
    add(choices, BorderLayout.NORTH);
    add(new JScrollPane(result),
      BorderLayout.CENTER);
    add(actions, BorderLayout.SOUTH);
  }
  JMenuBar createMenuBar() {
    JMenuBar bar = new JMenuBar();
    JMenu request = new JMenu("Request");
    request.setMnemonic('R');
    JMenu view = new JMenu("View");
    view.setMnemonic('V');
    for (String text : new String[]{"Preview",
        "Rename", "Clear", "Info"}) {
      JMenuItem item = new JMenuItem(text);
      item.setMnemonic(text.charAt(0));
      item.addActionListener(this);
      if ("Preview".equals(text)) {
        item.setAccelerator(KeyStroke.getKeyStroke(
          KeyEvent.VK_P,
          InputEvent.CTRL_DOWN_MASK));
      }
      if ("Info".equals(text)) { view.add(item); }
      else { request.add(item); }
    }
    bar.add(request); bar.add(view);
    return bar;
  }
  String summary() {
    if (subject.getSelectedValue() == null) {
      throw new IllegalArgumentException(
        "Choose a subject.");
    }
    if (!guide.isSelected() && !sheet.isSelected(
      )) {
```

Continue the same file on page 4. No imports or class header are repeated. After both pages, compile the whole folder.

WEDNESDAY continued | Step 1 complete replacement

Continue RequestDemo.java from page 3, left column then right. This is the remainder of the same file.

RequestDemo.java · lines 93–139

```
      throw new IllegalArgumentException(
        "Choose materials.");
    }
    if (!desk.isSelected() && !room.isSelected())
      {
      throw new IllegalArgumentException(
        "Choose a place.");
    }
    return subject.getSelectedValue() + ": Guide="
      + guide.isSelected()
      + "; Sheet=" + sheet.isSelected()
      + "; Place=" + (desk.isSelected() ? "Desk" :
        "Classroom");
  }
  void clearChoices() {
    subject.clearSelection();
    guide.setSelected(false); sheet.setSelected(
      false);
    desk.setSelected(true); result.setText(
      "Choices cleared.");
  }
  void acceptName(String input) {
    if (input == null) { return; }
    String clean = input.trim();
    if (clean.isEmpty() || clean.length() > 30) {
      result.setText(
        "Use 1-30 characters for the name.");
      return;
    }
    name = clean; result.setText("Named: " +
      name);
  }
  void clearIfConfirmed(int answer) {
    if (answer == JOptionPane.YES_OPTION) {
      clearChoices();
    } else if (answer == JOptionPane.NO_OPTION) {
      result.setText("Kept choices (No).");
    } else if (answer ==
      JOptionPane.CANCEL_OPTION) {
      result.setText("Kept choices (Cancel).");
    } else {
      result.setText("Kept choices (Closed).");
    }
  }
  @Override public void actionPerformed(
    ActionEvent e) {
    String command = e.getActionCommand();
```

RequestDemo.java · lines 140–186

```
    if ("Rename".equals(command)) {
      acceptName(JOptionPane.showInputDialog(this,
        "Request name (1-30 characters):", name));
      return;
    }
    if ("Clear".equals(command)) {
      clearIfConfirmed(
        JOptionPane.showConfirmDialog(this,
        "Clear subject and materials?",
        "Confirm clear",
          JOptionPane.YES_NO_CANCEL_OPTION));
      return;
    }
    try {
      String text = name + "\n" + summary();
      if ("Info".equals(command)) {
        JOptionPane.showMessageDialog(this, text,
          "Request information",
            JOptionPane.INFORMATION_MESSAGE);
      } else {
        result.setText(text);
      }
    } catch (IllegalArgumentException ex) {
      result.setText(ex.getMessage());
    }
  }
  public static void main(String[] args) {
    if (GraphicsEnvironment.isHeadless()) {
      System.out.println(
        "No display: GUI not opened.");
      return;
    }
    SwingUtilities.invokeLater(() -> {
        JFrame f = new JFrame(
          "School materials request");
        f.setDefaultCloseOperation(
          JFrame.DISPOSE_ON_CLOSE);
        RequestDemo panel = new RequestDemo();
        f.setContentPane(panel);
        f.setJMenuBar(panel.createMenuBar());
        f.setSize(640, 420);
        f.setMinimumSize(new Dimension(520, 380));
        f.setLocationRelativeTo(null);
        f.setVisible(true);
    });
  }
}
```

Output and evidence. The forced-headless launch printed exactly: No display: GUI not opened. A school-computer launch should show the existing controls plus Request and View menus. After Java + Guide + Desk, Preview should show My request followed by Java: Guide=true; Sheet=false; Place=Desk. This result was checked directly; window appearance and physical actions remain unverified.

Action	Required outcome
Request → Preview / Ctrl+P	Same validation and summary as Preview button.
Request → Rename	Cancel/close keeps the name; blank or >30 characters gives feedback; a valid trimmed name is used.
Request → Clear	Only YES_OPTION clears subject/materials and selects Desk. No/Cancel/close keeps choices. The name remains.
View → Info	Same validated current request in an information dialog; closing it does not change choices.

Activity 3 | Own-theme menus. Back up working sources. Add a menu bar with clearly named menus and only purposeful items linked to your existing actions/model. Choose distinct navigation letters. Record normal, invalid, corrected, declined and cancelled outcomes. Never treat “not No” as permission to destroy data. Use an explicit Yes test.
Check yourself: selecting a menu without adding a listener cannot perform the task. Reusing the same handler avoids different rules for buttons and menus. The example’s Clear keeps the request name; explain exactly what your own destructive action resets.
Read the safety rules in the listing: acceptName(null) returns without changing the name; invalid text leaves the name unchanged. clearIfConfirmed() alone authorizes clearing after Yes. Closing Info changes no request fields. These direct method outcomes were tested; a real dialog still needs the page 5 checks.
Own menu trace: item ____ → command ____ → model/rule ____ → feedback ____.
References: [1 N2], [2 YC], [3 KS], [4 MIT], [5 PA]. Links/excerpts: page 5. **Key validated facts:** A menu bar contains menus and menu items fire actions [N2]; cancellation should preserve progress [YC].
LIPLET | WEDNESDAY: After Step 1 and Activity 3 on pages 3–4, answer the WEDNESDAY strip (top of page 7). Detach only today’s strip.

THURSDAY | Complete and Repair Your Own Application

Goals: Compare requirements with actual results and choose repairs that preserve a useful task. **Value:** Report failed and unverified checks honestly and respond constructively to feedback. **Produce:** Complete the own-theme application, repair problems and record retest results.

Definition | Requirements versus results. A comparison between each promised behavior and what an actual trial showed. Compilation checks syntax and types; it cannot prove that a person can finish the task [YS] [YT].

ANALOGY Before calling a desk ready, compare each promise on its sign with what a user can actually complete.

Activity 4 | Complete your own-theme performance task. Use your Week 20 proposal and Monday plan. Finish one useful workflow, menus, checks, correction and safe cancellation. Use the task tools on page 6. Run the school-computer checks below, record failures, repair the responsible code once, then repeat the failed test and one surrounding successful test. Keep useful workers; do not introduce slow work into listeners.

Test record format: requirement ID | input/action | expected | actual | pass/fail/unverified | repair | retest. Record what happened, including messages; never copy an expected result into the actual column without running it. Classmate role-play is classroom feedback, not field research.

School-computer check	Record actual result; do not pre-mark Pass
Clicks / useful task	Open your launcher; complete the normal task by buttons and every menu item. Confirm the model result.
Keyboard / access	Use Tab/Shift+Tab and arrows/Space; try menu letters (example Alt+R/Alt+V), item letters and Ctrl+P. OS shortcuts can differ.
Resize / labels	Check the example at 520×380 and larger; verify labels, controls and scrolling remain reachable. Test your own minimum size.
Dialogs / cancellation	Info opens/closes; Rename Cancel/close, blank, 31 characters, valid trim; Clear No, Cancel and close all keep choices; only Yes clears.
Recovery / consistency	Make invalid input, correct it and repeat. Edit after Preview, then preview again; compare summary with current choices.
Responsiveness / workers	While Vehicle Check ready is running, move/resize the window and use controls; repeated clicks stay disabled until completion. Check your own workers if used.

Verification limitation: both starting/final Java versions compile with Java 8 compatibility. Headless launches and direct request/menu/worker checks passed. A normal launch failed with an X11 connection error. No working display or virtual display is available; live clicks, dialogs, keyboard, resizing and perceived responsiveness remain unverified. Check them at school before claiming completion.

Common-error guide

Actual message or check symptom	Meaning / correction
MenuError.java:2: error: cannot find symbol	A test class omitted Swing imports. Keep import javax.swing.*; at the top. The full real message names MenuError.java:2; file/line may vary.
IllegalArgumentException: Choose a subject.	Direct call to summary() had no subject. Select one subject and Preview again. The GUI handler shows Choose a subject. as feedback.
IllegalArgumentException: Choose materials.	Direct call had a subject but no material. Select Guide or Practice sheet; Preview again.
java.awt.AWTError: Can't connect to X11 window server using ':0' as the value of the DISPLAY variable.	This preparation could not reach its configured display. Run the GUI on the school desktop with a working display. A headless message does not establish GUI success.
Menu opens but no task runs. (observed symptom to check)	Missing listener or wrong action text. Use the same command and handler as the working button; retest both paths.
No/Cancel clears choices. (fault to check)	State changes occur before confirmation or branch is too broad. Call clearChoices() only when answer == JOptionPane.YES_OPTION. Retest all outcomes.

Opened university references

[YD] Yale University. Discovery. Excerpt: “real goals”. Identify the intended users and their task.

[YS] Yale University. Define the Scope. Excerpt: “success criteria”. Limit requirements to needs and available resources.

[YT] Yale University. User Testing. Excerpt: “complete tasks”. Observe task attempts and record problems before revision.

[YC] Yale University. Giving Users Control. Excerpt: “change their minds”. Explain consequences and allow safe changes of mind.

[N1] Nanyang Technological University. GUI Programming. Excerpt: “arranged from left-to-right”. Layouts arrange controls; listeners handle actions.

[N2] Nanyang Technological University. GUI Programming Part 2. Excerpt: “fires ActionEvent upon activation”. Menu items fire actions; confirmation returns a choice.

[KS] Kansas State University. Swing Event Dispatch Thread. Excerpt: “event handlers”. Keep event handlers short; start GUI work with invokeLater.

[MIT] Massachusetts Institute of Technology. Graphical User Interfaces. Excerpt: “view tree”. Separate interface actions from application rules; nest views.

[PA] University of Pennsylvania. Design and Usability Testing. Excerpt: “visible labels”. Use visible labels, clear actions and keyboard alternatives.

Source use: five numbered references per day appear in the daily reference lines and lesson log. Penn’s web-design advice is applied here only to labels, actions and keyboard alternatives. The classroom scenario and Java extension are teacher-created. Curriculum guide, pages 7–8: competency 7 includes combining components/layouts, menus and windows/dialogs. Existing parented dialogs cover the modest dialog scope. The course map remained inaccessible; nothing was inferred.

References: [1 YS], [2 YT], [3 YC], [4 KS], [5 MIT]. Links/excerpts: page 5. **Key validated facts:** Use actual task attempts to find problems [YT]; preserve safe correction during repairs [YC].

LIPLET | THURSDAY: After Activity 4 on pages 5–6, answer the THURSDAY strip (bottom of page 8). Detach only today’s strip.

FRIDAY | Verify Document and Present Your Application

Goals: Defend application scope and design using the proposal, code and actual test record. **Value:** Explain limitations honestly and respect a partner's feedback. **Produce:** Run consistency and usability checks, finish documentation and present the own-theme application.

Definition | Consistency and usability check. Consistency means the proposal, labels, code and evidence agree. Usability asks whether someone can complete the intended task with the interface [YT] [PA].

Activity 5 | Verify, document and present. Use your own application, code, proposal and test record. Ask a classmate to attempt the task without prompting; record actual steps and confusion as classroom feedback. Repair a genuine problem and retest. Submit sources/backups, plan, requirements/results checklist, test/repair record and short user notes: launcher, task, correction/cancel behavior and limits. Present for five minutes.

Worked-example limit: menus improve access to the existing draft task. They do not turn Preview into submission or implement Week 20's deferred save/load plan. Reopen after closing to observe the one-session limit; do not describe the draft as saved.

Own-theme performance task | 50 points

☐ Name own theme, users, problem and Week 20 proposal link; justify revisions.

☐ Link 3–5 limited requirements to needs and actual checks.

☐ Show a simple interface plan and normal/error/cancel user flow.

☐ Combine useful controls and nested layouts for one complete task.

☐ Connect actions to the existing model; keep checks, EDT updates and useful workers.

☐ Provide purposeful working menus, clear labels and keyboard access.

☐ Validate input, show useful feedback and allow correction.

☐ Keep choices on cancellation; require explicit Yes for destructive changes.

☐ Compare requirements with actual results; record repairs and retests.

☐ Document remaining limits and present the application honestly.

Plan before final coding

Plan	Your notes
User / task	_____
R__ → need → check	_____ _____
Panel/layout / flow	_____ _____

Starter with blanks

My existing theme is ____; users ____ need to ____ because ____. My proposal planned ____; I kept/revised ____ because ____. Requirements R1–R__ and needs: ____. Interface groups/layouts: ____. Flow: start ____ → action ____ → model ____ → result ____. Error/cancel path: ____. Menus/items/keys: ____. Retained worker (if needed): ____. Tests and actual results: ____. Repair/retest: ____. Existing ____; built and checked ____; planned/deferred ____; still unverified ____.

Self-check

Check	Status
One useful task matches proposal/revision?	Yes / Not yet
Model, controls and menus agree?	Yes / Not yet
Errors correctable; cancel safe; Yes explicit?	Yes / Not yet
Keyboard, resize and worker checks recorded?	Yes / Not yet
Every requirement has actual evidence or an unverified label?	Yes / Not yet
Limits, repairs and documentation honest?	Yes / Not yet

Two three-hour days

Thursday: 10 min recall + 15 min requirements review + 90 min completion + 35 min test/repair + 15 min notes + 15 min slip = 180 min.

Friday: 10 min recall + 30 min consistency/usability + 25 min repairs + 20 min documentation + 20 min rehearsal + 60 min presentations + 15 min slip = 180 min.

Requirements versus results

ID / expected	Actual / repair / retest
R__ / _____	_____ _____
R__ / _____	_____ _____
R__ / _____	_____ _____

Use notebook rows for every requirement. Mark unchecked behavior **unverified**.

Five-minute presentation

• 30 s: own theme, user, problem and proposal/revision.

• 1 min: complete one useful task and show the result.

• 2 min: bad input, correction and safe cancellation; show actual test evidence.

• 1 min: point to menu wiring, model call and safe interface/worker code.

• 30 s: answer a question and state one remaining limit.

Large class: circles of at most 12 allow 12 × 5 min = 60 min. Peer listeners keep evidence notes; the teacher rotates and reviews every application and record.

Prepare using your own work

1. Which own requirement solves the user's problem?
2. Where does your own listener call the existing model?
3. What happens when a user cancels or closes a dialog?
4. Which actual test failed, what changed, and what did the retest show?
5. Which capability remains planned or unverified?

Six-criterion rubric

Criterion	Pts	Score
Need proposal and limited design	8	___
Useful workflow layouts and model connection	12	___
Purposeful menus labels and keyboard access	8	___
Validation correction cancellation and confirmation	10	___
Actual tests repairs and honest limitations	8	___
Documentation and clear presentation	4	___
TOTAL	50	___

Full points: complete and evidenced; about half: partly working or weak evidence; zero: absent/unrelated. Award intermediate whole points. Do not reward extra controls or feature count. The task score includes presentation; slip scores are separate.

References: [1 YT], [2 PA], [3 YS], [4 KS], [5 MIT]. Links/excerpts: page 5. **Key validated facts:** Observe task completion [YT]; check labels and keyboard alternatives [PA].

LIPLET | FRIDAY: After Activity 5 on page 6 and your presentation, answer the FRIDAY strip (top of page 8). Detach only today's strip.

WEDNESDAY | Add Purposeful Menus Safely

DEC 9 · Answer after Step 1 and Activity 3 on pages 3–4.
Name: _____ Section: _____ Score: ____ / 15

3.1 Define (5 pts): After Step 1 and Activity 3, explain JMenuBar, JMenu and JMenuItem using your menu plan.

3.2 Explain (5 pts): In Step 1, why do menu items use the same listener as buttons? Explain the exact Yes condition and what No/Cancel/close preserve.

3.3 Connect (5 pts): Connect Activity 3 to your own existing model and exception handling: trace a menu action, a model rule and useful feedback.

CUT ON TUESDAY · DETACH THE TUESDAY STRIP BELOW

TUESDAY | Combine a Complete User Workflow

DEC 8 · Answer after Activity 2 on page 2.
Name: _____ Section: _____ Score: ____ / 20

2.1 Define (5 pts): In Activity 2, explain integration and the roles of a nested panel, model method and listener in your workflow.

2.2 Explain (5 pts): Trace your Activity 2 input → action → model → result; explain invalid input and how the user corrects it.

2.3 Identify (5 pts): For Activity 2, identify four parts in your own code: (a) input control ___; (b) nested panel/layout ___; (c) model call ___; (d) feedback update ___.

2.4 Connect (5 pts): Connect Activity 2 to exceptions or threads: explain a retained validation rule or worker and why removing it would hurt your task.

CUT ON MONDAY · DETACH THE MONDAY STRIP BELOW

MONDAY | Plan One Useful Application

DEC 7 · Answer after Activity 1 on page 1.
Name: _____ Section: _____ Score: ____ / 15

1.1 Define (5 pts): In Activity 1, distinguish existing, planned, completed and unverified capabilities using your own proposal and program.

1.2 Explain (5 pts): For Activity 1, trace one user need through your limited requirement, interface plan and user flow to a check.

1.3 Connect (5 pts): Connect Activity 1 to Weeks 11–20: name an existing model method and a proposal finding that support your chosen task.

FRIDAY | Verify Document and Present Your Application

DEC 11 · Answer after Activity 5 on page 6 and your presentation.
Name: _____ Section: _____ Score: ____ / 20

5.1 Point (5 pts): Point to your own application's menu wiring and model call. Link them to one requirement in your proposal and one actual test record.

5.2 Evaluate (5 pts): Defend one design or scope choice in your own application using the proposal, code and actual usability/test evidence; state a limit.

5.3 Apply (5 pts): A user cancels a destructive action in your own application. Trace the safe path in your code and add a test tied to your requirement; record the actual result if run, otherwise mark it unverified.

5.4 Reflect (5 pts): Compare your own Week 20 proposal with the application you presented. Explain one revision or repair, its actual evidence and what remains planned or unverified.

CUT ON THURSDAY · DETACH THE THURSDAY STRIP BELOW

THURSDAY | Complete and Repair Your Own Application

DEC 10 · Answer after Activity 4 on pages 5–6.
Name: _____ Section: _____ Score: ____ / 15

4.1 Define (5 pts): In Activity 4, distinguish a requirement, an expected result, an actual result and an unverified check using your own application.

4.2 Explain (5 pts): Describe one own-theme failed check, the repair, and actual retest evidence. If none failed, name a tested risk and one remaining unverified check.

4.3 Connect (5 pts): Connect Activity 4 to your Week 20 proposal and earlier Java work: justify one scope revision while retaining a useful model rule or worker.

Week 21 CodeShare practice additions

Use this appendix with the original module. Original lesson pages and sample listings remain unchanged. Text backups and data remain unchanged in the supporting-files folder.

Console checks exercise component state, layout or event behavior without opening a window. The message No display: GUI not opened. alone never completes an alteration. Check the actual interface on a school computer; browser GUI rendering and interaction remain unverified.

Run the unchanged sample first. Sample verified records the demonstration only. Make Your task changes, then compare the complete Expected task output. A successful alteration earns Completed. Keep module work, personal work and submitted versions separate. Paper liplets remain teacher assessed.

Step 1 alteration

Sample: java-2-w21-d3-s1 Task: java-2-w21-d3-s1-alter-v1

Add a Reset menu item after the existing Request menu items. Set its action command to "Clear", register the existing listener and add it to the Request menu. Keep Clear confirmation unchanged. Print the actual menu count and the new item action command. At school, also check Yes, No, Cancel and close through Reset.

Before the existing headless check in `RequestDemo.java`, add this console check. Keep the original sample message and the window code.

```
JMenu request = new RequestDemo().createMenuBar().getMenu(0);
System.out.println("Request items: " + request.getItemCount());
System.out.println("Reset action: " + request.getItem(3).getActionCommand());
```

Expected task output

```
Request items: 4
Reset action: Clear
No display: GUI not opened.
```